COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	Aug 18, 2026	**DATE SUBMITTED:**	Aug 27, 2026
NAME:	Ellaiza C. Vizcayno	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 6 **Searching Algorithms**

1. PROCEDURE OUTPUT

Screenshot

main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib> //for generating random integers
#include <time.h> //will be used for our seeding function
#include "node.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main(){
    //generate random values
    int dataset[max_size];
    srand(time(0));
    for(int i = 0; i < max_size; i++){
        dataset[i] = rand();
    }
    //show your datasets content
    for(int i = 0; i < max_size; i++ ){
        std::cout << dataset[i] << " ";
        if((i+1) % 5 == 0 ){ //for easy readability
            std::cout << std::endl;
        }
    }

    return 0;
}
```

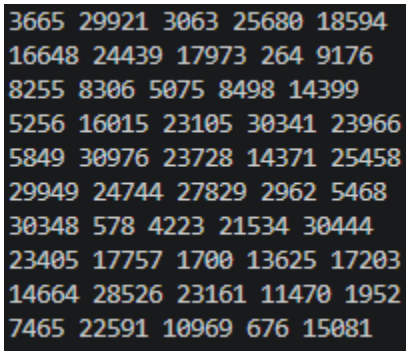
	
Observations	While going through this code, one thing that stood out to me was the use of srand(time(0)) for seeding the random number generator. Since time(0) returns the current time in seconds, and time is always changing, this means the seed will be different every time the code is run, which is why the random values generated also change each run. This made me wonder why we specifically use srand(time(0)) instead of just leaving srand() out entirely. I found out that if srand() is never called, it behaves as if srand(1) was called by default, so the program would technically still work, but it would generate the same sequence of "random" numbers every single time it runs, since the seed would always be 1. This also made me curious why the default isn't srand(0) instead, especially since 0 is usually the value we associate with starting points, like how we use 0 as the starting index for arrays. However, I found out that this is actually intentional, since some rng algorithms can produce degenerate results when seeded with 0, meaning the sequence can get stuck or repeat poorly instead of behaving randomly. Using 1 as the default seed instead avoids this issue, which is why 1 was chosen as the safer default instead of 0. From this, I also observed that if we want a fixed randomly generated set of elements for an array, we can simply replace time(0) with a number instead, since a fixed seed will always produce the same sequence rather than a different one each run. I also added a modulo condition in the display loop to print a new line after every 5 elements. I did this so I could easily confirm that the array actually contained 50 values, since counting them in rows of 5 was much quicker than checking one long unbroken line of numbers.

Table 6-1. Data Generated and Observations

Code

```
6   template <typename T>
7   void linearSearchArr(int arrSize, T dataFind, T list[]){
8       for(int i = 0; i < arrSize; i++){
9           if (dataFind == list[i]){
10              std::cout << "Searching is successful!" << std::endl;
11              return;
12          }
13      }
14      std::cout << "Searching is unsuccessful.." << std::endl;
15  }
```

```
std::cout << std::endl;
int userInput;
std::cout << "What number to search? : ";
std::cin >> userInput;
std::cout << "Array result LS: ";

linearSearchArr(max_size, userInput, dataset);
```

Output

```
8339 17889 1234 27808 29636
23902 13174 2218 4613 27835
823 27417 5088 4337 10625
3771 30653 16984 8856 3793
31653 22367 31104 24421 17484
8064 18202 24542 30031 10867
11407 31697 7866 4002 19697
19994 9165 18910 2592 31276
32512 31568 2283 12197 1860
11698 16504 3110 6796 25279

What number to search? : 4002
Array result LS: Searching is successful!
```

Observations

The template function linearSearchArr takes in the max size of the array, the data we are trying to find, and the array itself as its parameters. Inside the function, a for loop is used to traverse the array, starting from the first index and continuing until it reaches the max size. As the loop goes through the array, it compares each element to the data we are finding. There is an if condition that checks if the current element matches the data we are looking for, and if it does, the program prints out that searching is successful. But if the for loop finishes traversing the whole array without finding a match, the program prints out that searching is unsuccessful instead.

Table 6-2a. Linear Search for Arrays

Code

```cpp
template <typename T>
void linearSearchLL(Node<T> *head, T dataFind){
    while (head != nullptr){
        if(head->data == dataFind){
            std::cout << "Searching is successful!" << std::endl;
            return;
        }
        head = head->next;
    }
    std::cout << "Searching is unsuccessful.." << std::endl;
}
```

```cpp
Node<char>* name1 = new_node('E');
Node<char>* name2 = new_node('L');
Node<char>* name3 = new_node('L');
Node<char>* name4 = new_node('A');
Node<char>* name5 = new_node('I');
Node<char>* name6 = new_node('Z');
Node<char>* name7 = new_node('A');

name1->next = name2;
name2->next = name3;
name3->next = name4;
name4->next = name5;
name5->next = name6;
name6->next = name7;

listTraversal(name1);

char userInput;

std::cout << std::endl;
std::cout << "What char to search for? : ";
std::cin >> userInput;
std::cout << "Linked List result LS: "<< std::endl;
linearSearchLL(name1, userInput);
```

Output

```
E->L->L->A->I->Z->A
What char to search for? : Z
Linked List result LS:
Searching is successful!
```

Observations	This linear search using a linked list works the same way the array version does. It goes through the list while comparing the data in each node to the data we are looking for. In this case, it uses a while loop that will continue to compare values as long as the head does not equal nullptr. That is where the if condition comes in to catch the match, and once it finds the data, it prints that searching is successful. If not, the head node is passed onto the next node and the comparison and passing of node repeats. Once it reaches the end of the list without finding a match, it prints that searching is unsuccessful.

Table 6-2b. Linear Search for Linked List

Code

```
45  template <typename T>
46  void binarySearchArr(int arrSize, T dataFind, T list[]){
47      int low = 0, up = arrSize - 1;
48
49      while (low <= up){
50          int mid = int((low + up) / 2);
51          if (dataFind == list[mid]){
52              std::cout << "Searching is successful! Found at index: " << mid << std::endl;
53              return;
54          }
55          if (dataFind < list[mid]){
56              up = mid - 1;
57          }else{
58              low = mid + 1;
59          }
60      }
61      std::cout << "Searching is unsuccessful.." << std::endl;
62  }
```

```
int dataset[max_size];
srand(67);
dataset[0] = rand() % 100;
for (int i = 1; i < max_size; i++){
    dataset[i] = dataset[i - 1] + (rand() % 100 + 1);
}
for(int i = 0; i < max_size; i++ ){
    std::cout << dataset[i] << " ";
    if((i+1) % 5 == 0 ){
        std::cout << std::endl;
    }
}

std::cout << std::endl;
int userInput;
std::cout << "What number to search? : ";
std::cin >> userInput;
std::cout << "Array result BS: ";
binarySearchArr(max_size, userInput, dataset);
```

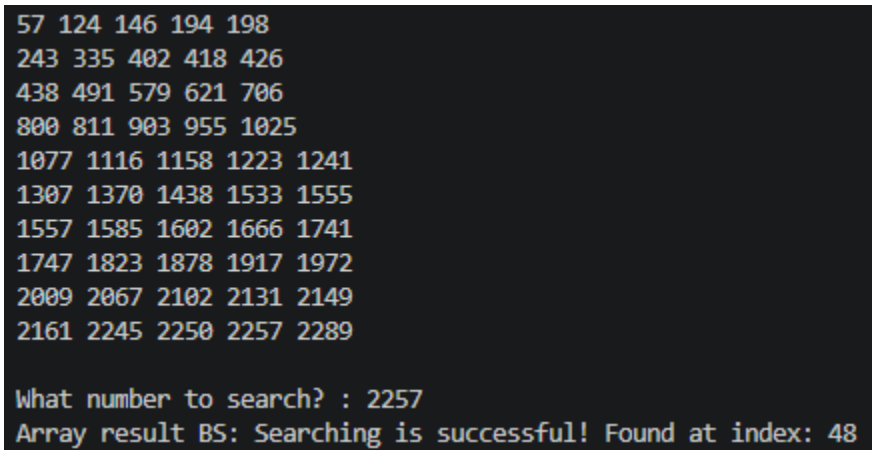
Output	
Observations	Something to note is that I did not want to hard code an array of ordered elements, so I made use of generating random elements, but I needed to find a way to make it ordered. So I decided to make the first element start off as a small value by having rand() modulo 100, this keeps the value between 0 and 99 instead of a huge random number. For the rest of the array, I inserted each value by taking the previous element and adding another random value, also modulo 100 plus 1, so that the number added is always at least 1. This way, each new value is guaranteed to be bigger than the one before it, which keeps the whole array in ascending order without me having to manually type out the values myself. One thing I noticed while running the binary search is that it found the value I was searching for a lot faster than linear search would have, since it does not need to check every single element one by one. Instead, it starts at the middle of the array and compares that value to what I am looking for, and depending on whether my target is higher or lower, it eliminates half of the array each time instead of checking it. This also made me realize why the array needed to be in order in the first place, since binary search only works correctly if it can safely assume that everything to the left of the middle is smaller and everything to the right is bigger, which is exactly why I had to make sure the array was sorted before this part of the code could even work properly.

Table 6-3a. Binary Search for Arrays

Code

```
64  template <typename T>
65  void binarySearchLL(Node<T>* start, T dataFind){
66      Node<T>* last = nullptr;
67      while (start != last){
68          Node<T>* mid = getMiddle(start, last);
69          if (mid->data == dataFind){
70              std::cout << "Searching is successful!" << std::endl;
71              return;
72          }else if (mid->data > dataFind){
73              last = mid;
74          }else if (mid->data < dataFind){
75              start = mid->next;
76          }
77      }
78      std::cout << "Searching is unsuccessful.." << std::endl;
79  }
```

```
29  template <typename T>
30  Node<T>* getMiddle(Node<T>* start, Node<T>* last){
31      Node<T>* slow = start;
32      Node<T>* fast = start;
33
34      while (fast != last && fast->next != last){
35          //move slow one step
36          slow = slow->next;
37
38          //move fast two steps
39          fast = fast->next->next;
40      }
41
42      return slow;
43  }
```

```cpp
template <typename T>
Node<T>* createOrderedList(int size){
    Node<T>* head = nullptr;
    Node<T>* tail = nullptr;

    int value = rand() % 100;

    for(int i = 0; i < size; i++){
        Node<T>* newNode = new_node(value);
        if(head == nullptr){
            head = newNode;
            tail = newNode;
        } else {
            tail->next = newNode;
            tail = newNode;
        }
        value += (rand() % 100 + 1);
    }
    return head;
}
```

```cpp
int size;
int userInput;
std::cout << "Creating an ordered linked list\n"
          << "composed of randomly generated values: \n"
          << "Enter size of linked list: ";
std::cin >> size;
Node<int>* head = createOrderedList<int>(size);
listTraversal(head);
std::cout << "\nLinked List has been created!\n"
          << "What to search for? : ";
std::cin >> userInput;
binarySearchLL(head, userInput);
```

Output

```
Creating an ordered linked list
composed of randomly generated values:
Enter size of linked list: 7
41->109->144->145->215->240->319
Linked List has been created!
What to search for? : 319
Searching is successful!
```

Observations	I noticed that for array, we can immediately find the middle of the list by dividing it by 2, but for a linked list we need a whole new function to calculate the middle for us. I find the method of calculating the middle so interesting because it uses a slow and fast pointer. The slow pointer moves one node at a time, while the fast pointer moves two nodes at a time, and since the fast pointer is moving twice as quickly, by the time it reaches the end of the list, the slow pointer will only have reached the halfway point. This is why the loop keeps checking if fast has not reached last and if fast->next has not reached last either, since it needs to stop the moment fast catches up so that slow ends up exactly at the middle. After that, it is pretty much the same thing as array binary search. We compare the middle node's data to the value we are looking for, and if it matches, searching is successful. If not, we check whether the value we are looking for is higher or lower than the middle. Although unlike the array version where we just adjust the low or up index, for linked list we adjust the start or last pointer instead, and every time this happens, getMiddle has to be called again to recalculate the new middle based on the updated start or last, since there is no direct index we can just divide. I also made another randomly generated linked list, but this time I made it so that the values are inserted in order automatically, similar to how I did it with the array. I started with a small random value using rand() modulo 100, and for every new node after that, I added the previous value to another random value, also modulo 100 plus 1, so that each new node in the list is guaranteed to be bigger than the one before it. This meant that instead of hard coding a sorted linked list myself, the createOrderedList function builds it as it goes, and by the time it finishes, the entire linked list is already in ascending order, which allows binary search to work correctly on it.

Table 6-3b. Binary Search for Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem	Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?
Code	

```cpp
#include <iostream>
#include <cstdlib> //for generating random integers
#include <time.h> //will be used for our seeding function
#include "node.h"
#include "searchingAlgo.h"

// const int max_size = 50;
int main(){
    int problem1[10] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    Node<int>* head = createList<int>();

    std::cout << "Array LS Result: \n";
    linearSearchArr(10, 18, problem1);
    std::cout << std::endl;
    std::cout << "Linked List LS Result: \n";
    linearSearchLL(head, 18);


    return 0;
}
```

```cpp
template <typename T>
Node<T>* createList(){
    char choice = 'y'; int count = 1; int newData;
    Node<T> *temp, *head, *node;
    while (choice == 'y'){
        std::cout << "Enter data: ";
        std::cin >> newData;
        if (count==1){
            head = new_node(newData);
            std::cout << "Successfully added " << head->data << " to the list.\n";
            count++;
        } else if (count==2) {
            node = new_node(newData);
            head->next = node;
            node->next = nullptr;
            std::cout << "Successfully added " << node->data << " to the list.\n";
            count++;
        } else {
            temp = head;
            while (true){
                if(temp->next == nullptr) break;
                temp = temp->next;
            }
            node = new_node(newData);
            temp->next = node;
            std::cout << "Successfully added " << node->data << " to the list.\n";
            count++;
        }
        //when to end
        std::cout << "Continue? (y/n)";
        std::cin >> choice;
        if (choice == 'n') break;
    }
    return head;
}
```

```cpp
template <typename T>
void linearSearchArr(int arrSize, T dataFind, T list[]){
    for(int i = 0; i < arrSize; i++){
        int j = i;
        std::cout << "Count: " << j+1 << std::endl;
        if (dataFind == list[i]){
            std::cout << "Searching is successful!" << std::endl;
            return;
        }
    }
    std::cout << "Searching is unsuccessful.." << std::endl;
}

template <typename T>
void linearSearchLL(Node<T> *head, T dataFind){
    int i = 0;
    while (head != nullptr){
        i++;
        std::cout << "Count: " << i << std::endl;
        if(head->data == dataFind){
            std::cout << "Searching is successful!" << std::endl;
            return;
        }
        head = head->next;
    }
    std::cout << "Searching is unsuccessful.." << std::endl;
}
```

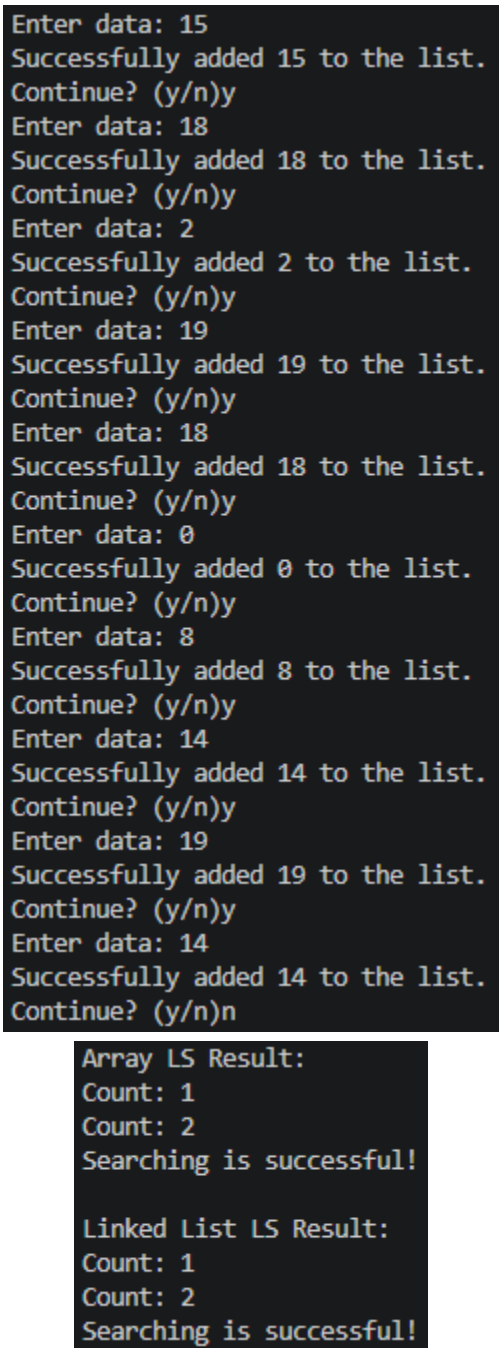
Output	
Question	How many comparisons would be necessary to find the key ‘18’?
Answer	According to the results, it took **2 comparisons** to find key ‘18’.

Table 7-1. Problem 1

Problem

Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element 'k'. Test on the same list given in problem 1.

Code

```cpp
#include <iostream>
#include <cstdlib> //for generating random integers
#include <time.h> //will be used for our seeding function
#include "node.h"
#include "searchingAlgo.h"

// const int max_size = 50;
int main(){
    int problem1[10] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    Node<int>* head = createList<int>();
    std::cout << "\n\n";
    int k;

    std::cout << "What to search for? : ";
    std::cin >> k;

    std::cout << "Array LS Result: \n";
    linearSearchArr(10, k, problem1);
    std::cout << std::endl;
    std::cout << "Linked List LS Result: \n";
    linearSearchLL(head, k);


    return 0;
}
```

Output

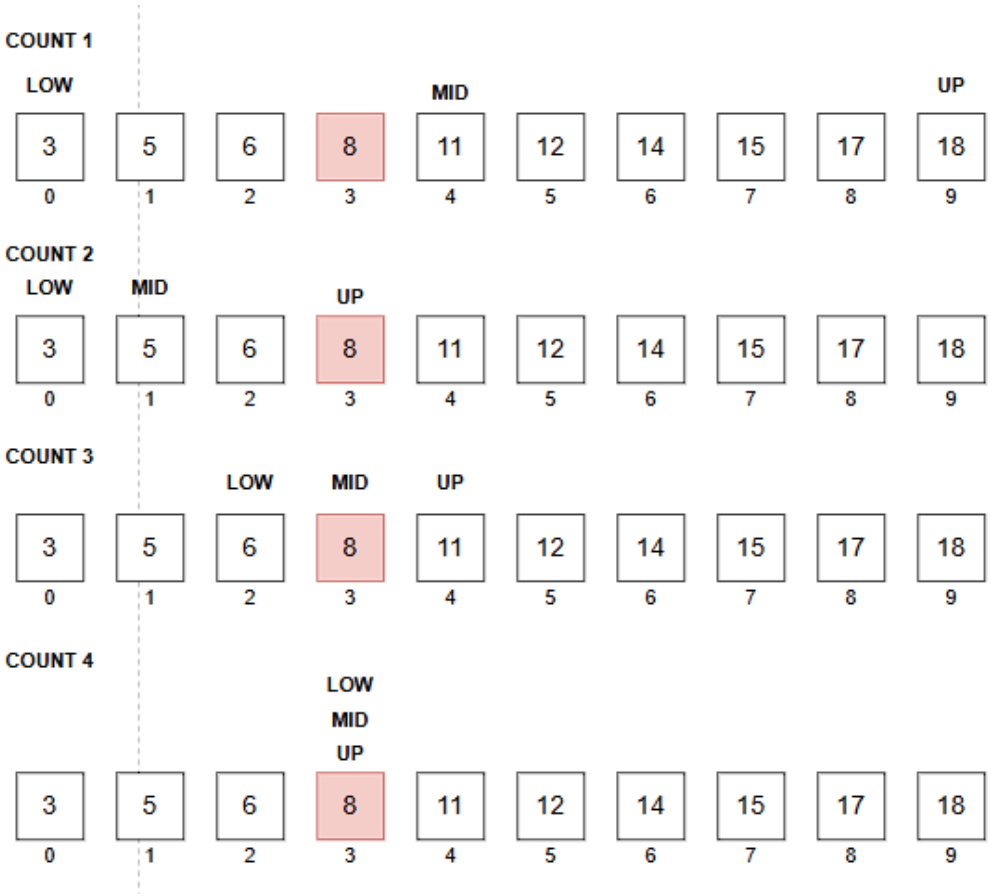
```
What to search for? : 14
Array LS Result:
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
Count: 6
Count: 7
Count: 8
Searching is successful!
```

```
Linked List LS Result:
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
Count: 6
Count: 7
Count: 8
Searching is successful!
```

Answer

In searching for key '14', it took **8 comparisons**.

Table 7-2. Problem 2

Problem	Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and output console.
Drawing	

Code

```
// PROBLEM 3
Node<int>* node1 = new_node(3);
Node<int>* node2 = new_node(5);
Node<int>* node3 = new_node(6);
Node<int>* node4 = new_node(8);
Node<int>* node5 = new_node(11);
Node<int>* node6 = new_node(12);
Node<int>* node7 = new_node(14);
Node<int>* node8 = new_node(15);
Node<int>* node9 = new_node(17);
Node<int>* node10 = new_node(18);

node1->next = node2;
node2->next = node3;
node3->next = node4;
node4->next = node5;
node5->next = node6;
node6->next = node7;
node7->next = node8;
node8->next = node9;
node9->next = node10;
Node<int>* head = node1;

int problem3[10] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
std::cout << "Array BS Result: \n";
binarySearchArr(10, 8, problem3);
std::cout << "Linked List BS Result: \n";
binarySearchLL(head, 8);
```

```cpp
template <typename T>
void binarySearchArr(int arrSize, T dataFind, T list[]){
    int low = 0, up = arrSize - 1;
    int count = 0;

    while (low <= up){
        int mid = int((low + up) / 2);
        count++;
        std::cout << "\nCount: " << count
                  << "\nLow: Index " << low
                  << "\nMid: Index " << mid
                  << "\nUp: Index " << up << std::endl;
        if (dataFind == list[mid]){
            std::cout << "Searching is successful! Found at index: " << mid << std::endl;
            return;
        }
        if (dataFind < list[mid]){
            up = mid - 1;
        }else{
            low = mid + 1;
        }
    }
    std::cout << "Searching is unsuccessful.." << std::endl;
}
```

```cpp
template <typename T>
void binarySearchLL(Node<T>* start, T dataFind){
    Node<T>* last = nullptr;
    int count = 0;
    while (start != last){
        count++;
        Node<T>* mid = getMiddle(start, last);
        std::cout << "\nCount: " << count
                  << "\nLow: " << start->data
                  << "\nMid: " << mid->data;
        if (last != nullptr){
            std::cout << "\nUp: " << last->data << std::endl;
        }else {
            std::cout << "\nUp: nullptr" << std::endl;
        }
        if (mid->data == dataFind){
            std::cout << "Searching is successful!" << std::endl;
            return;
        }else if (mid->data > dataFind){
            last = mid;
        }else if (mid->data < dataFind){
            start = mid->next;
        }
    }
    std::cout << "Searching is unsuccessful.." << std::endl;
}
```

Output

```
Array BS Result:

Count: 1
Low: Index 0
Mid: Index 4
Up: Index 9

Count: 2
Low: Index 0
Mid: Index 1
Up: Index 3

Count: 3
Low: Index 2
Mid: Index 2
Up: Index 3

Count: 4
Low: Index 3
Mid: Index 3
Up: Index 3
Searching is successful! Found at index: 3
```

```
Linked List BS Result:

Count: 1
Low: 3
Mid: 12
Up: nullptr

Count: 2
Low: 3
Mid: 6
Up: 12

Count: 3
Low: 8
Mid: 11
Up: 12

Count: 4
Low: 8
Mid: 8
Up: 11
Searching is successful!
```

Table 7-3. Problem 3

Problem	Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.
Code	

```
// PROBLEM 3 & 4
Node<int>* node1 = new_node(3);
Node<int>* node2 = new_node(5);
Node<int>* node3 = new_node(6);
Node<int>* node4 = new_node(8);
Node<int>* node5 = new_node(11);
Node<int>* node6 = new_node(12);
Node<int>* node7 = new_node(14);
Node<int>* node8 = new_node(15);
Node<int>* node9 = new_node(17);
Node<int>* node10 = new_node(18);

node1->next = node2;
node2->next = node3;
node3->next = node4;
node4->next = node5;
node5->next = node6;
node6->next = node7;
node7->next = node8;
node8->next = node9;
node9->next = node10;
Node<int>* head = node1;

const int size = 10;
int problem3[size] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
std::cout << "Array BS Result: \n";
binarySearchArr(size, 8, problem3);
std::cout << "Array BS Recursive Result: \n";
binarySearchArrRecursive(problem3, 8, 0, size-1);

std::cout << "Linked List BS Result: \n";
binarySearchLL(head, 8);
std::cout << "Linked List BS Recursive Result: \n";
binarySearchLLRecursive<int>(head, nullptr, 8);
```

```cpp
template <typename T>
void binarySearchArrRecursive(T list[], T dataFind, int low, int up){
    if (low > up){
        std::cout << "Searching is unsuccessful.." << std::endl;
        return;
    }
    int mid = (low + up) / 2;
    if (dataFind == list[mid]){
        std::cout << "Searching is successful! Found at index: " << mid << std::endl;
        return;
    }
    if (dataFind < list[mid]){
        binarySearchArrRecursive(list, dataFind, low, mid - 1);
    } else {
        binarySearchArrRecursive(list, dataFind, mid + 1, up);
    }
}
```

```cpp
template <typename T>
void binarySearchLLRecursive(Node<T>* start, Node<T>* last, T dataFind){
    if (start == last){
        std::cout << "Searching is unsuccessful.." << std::endl;
        return;
    }
    Node<T>* mid = getMiddle(start, last);
    if (mid->data == dataFind){
        std::cout << "Searching is successful!" << std::endl;
        return;
    } else if (mid->data > dataFind){
        binarySearchLLRecursive(start, mid, dataFind);
    } else {
        binarySearchLLRecursive(mid->next, last, dataFind);
    }
}
```

Output

```
Array BS Result:

Count: 1
Low: Index 0
Mid: Index 4
Up: Index 9

Count: 2
Low: Index 0
Mid: Index 1
Up: Index 3

Count: 3
Low: Index 2
Mid: Index 2
Up: Index 3

Count: 4
Low: Index 3
Mid: Index 3
Up: Index 3
Searching is successful! Found at index: 3
Array BS Recursive Result:
Searching is successful! Found at index: 3
```

```
Linked List BS Result:

Count: 1
Low: 3
Mid: 12
Up: nullptr

Count: 2
Low: 3
Mid: 6
Up: 12

Count: 3
Low: 8
Mid: 11
Up: 12

Count: 4
Low: 8
Mid: 8
Up: 11
Searching is successful!
Linked List BS Recursive Result:
Searching is successful!
```

Table 7-4. Problem 4

Conclusion

In this activity I learned how to create and understand searching algorithms. These searching algorithms have linear/sequential search and binary search. Linear/sequential search is searching and comparing of data in an unordered array or list one at a time, from index 0 to the max. Binary search on the other hand, searches for data in an ordered list, its index 0 starting at the lowest value ascending as the index grows larger. This search begins at the middle, and it compares whether the value being compared is lower or higher than the value in the middle. If it determines the value to be lower than the middle, it narrows the search to the lower half by moving up down to just before the middle, and if the value is higher instead, it narrows the search to the upper half by moving low up to just after the middle. This keeps repeating on a smaller part of the list each time, until the value is found or there is nothing left to search, which is what makes binary search a lot faster than linear search for big, ordered lists.

For the procedure, it was creating the different implementations of linear/sequential and binary search using arrays and linked lists, while each performing the exact same steps, its internal structure varies. The biggest difference is that with an array, getting to the middle is instant, since low and up can just be divided to go straight to that index. But with a linked list, there is no way to jump to an index like that, the nodes actually have to be walked through one by one to even get there, which is why a whole separate function like getMiddle had to be made using a slow and fast pointer. This means that even though binary search is doing the exact same comparisons either way, the array version is naturally faster to reach the middle each time, while the linked list version has to do extra work just to arrive at the same starting point.

For the supplementary, it focuses on challenging the understanding of linear and binary search using problems. Looking at problem 3 and 4, the iterative and recursive binary search ended up needing the exact same number of comparisons to find the value, since the logic itself did not actually change, it was just written differently. The recursive version does not loop back like the iterative one does, instead it calls itself again with a smaller part of the list each time, so instead of one function handling everything through a while loop, multiple calls end up stacking on top of each other until the answer is found or the list runs out to check. So even though the recursive version looks different and is structured differently, it is really just doing the same amount of work as the iterative one, just spread across separate function calls instead of one loop.

To conclude, I think I did well in this activity. This seemed really easy for me to understand and complete. For improvement, maybe I could have formatted the code output and maybe improve how I name my functions and variables. Overall it was fun and simple, and just the right amount.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **grammar correction, summarization, code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI in improving my analysis, observations, and conclusion. It was also used to help give me ideas in tackling problems without taking over the whole activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.